

API Reference

1. Overview

KneeVision exposes two services. The Node.js/Express backend handles user accounts, file uploads, MongoDB persistence, and proxying to the ML service. The FastAPI ML service runs the EfficientNet-B4 model, generates Grad-CAM heatmaps, performs similarity search against the SQLite reference database, and hosts the quiz data. All backend routes are under `/api/v1`. ML service routes are on its own host.

1.1 Authentication

Most backend endpoints require a JWT. Get one from the login endpoint and include it as:

```
http
Authorization: Bearer <your-token-here>
```

Tokens expire after one hour. On receiving a 401, prompt the user to log in again.

1.2 HTTP Status Codes

Code	Meaning
200 OK	Success.
201 Created	Resource created.
400 Bad Request	Missing or invalid request fields.
401 Unauthorized	Token missing, expired, or invalid.
404 Not Found	Resource not found or belongs to another user.
409 Conflict	Duplicate — e.g. email already registered.
500 Server Error	Unexpected internal error.

1.3 Upload Flow

Image uploads follow a three-step sequence that must be performed in order:

- `POST /api/v1/uploads/presign` — returns `uploadUrl` and `fileUrl`.
- `PUT /api/v1/uploads/:key` — upload raw bytes to the returned `uploadUrl`.
- `POST /api/v1/images` — register the image in MongoDB with the returned `fileUrl`.

2. Authentication

POST /api/v1/auth/register

Create a new user account. (No auth)

Request body

Field	Type	Required	Description
email	string	Yes	Valid email. Stored lowercase.
password	string	Yes	Minimum 8 characters.

Code snippets

```
curl
curl -X POST http://localhost:4000/api/v1/auth/register \
-H "Content-Type: application/json" \
-d '{"email": "student@pace.edu", "password": "securepass123"}'
```

Response — 201 Created

```
json
{
  "id": "64f1a2b3c4d5e6f7a8b9c0d1",
  "userId": "64f1a2b3c4d5e6f7a8b9c0d1",
  "email": "student@pace.edu",
  "createdAt": "2026-05-01T10:00:00.000Z"
}
```

POST /api/v1/auth/login

Log in and receive a JWT token. (No auth)

Request body

Field	Type	Required	Description
email	string	Yes	Registered email.
password	string	Yes	Account password.

Code snippets

```
curl
curl -X POST http://localhost:4000/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email": "student@pace.edu", "password": "securepass123"}'
```

Response — 200 OK

```
json
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "64f1a2b3c4d5e6f7a8b9c0d1",
    "email": "student@pace.edu",
    "createdAt": "2026-05-01T10:00:00.000Z"
  }
}
```

3. Images

Non-DICOM images are automatically preprocessed on upload: converted to grayscale PNG and resized to fit within 1024×1024 (aspect ratio preserved, no upscaling). DICOM files are stored as-is.

POST /api/v1/uploads/presign

Step 1 — get an upload URL and file key. (No auth)

Request body

Field	Type	Required	Description
filename	string	Yes	Original filename including extension.
contentType	string	Yes	MIME type, e.g. image/jpeg or application/dicom.

Code snippets

```
curl
curl -X POST http://localhost:4000/api/v1/uploads/presign \
-H "Content-Type: application/json" \
-d '{"filename":"knee.jpg","contentType":"image/jpeg"}'
```

Response — 200 OK

```
json
{
  "uploadUrl": "/api/v1/uploads/1714555200000-knee.png",
  "method": "PUT",
  "headers": {
    "Content-Type": "image/jpeg"
  },
  "fileUrl": "/uploads/1714555200000-knee.png"
}
```

PUT /api/v1/uploads/:key

Step 2 — upload raw file bytes. (No auth)

Content-Type must match the value returned in the presign headers object.

Path parameter

Param	Type	Description
:key	string	URL-encoded filename from the presign uploadUrl.

Code snippets

```
curl
curl -X PUT http://localhost:4000/api/v1/uploads/1714555200000-knee.png \
-H "Content-Type: image/jpeg" \
--data-binary @knee.jpg
```

Response — 200 OK

```
json
{
  "ok": true
}
```

POST /api/v1/images

Step 3 — register image in MongoDB. (Auth required)

Request body

Field	Type	Required	Description
fileUrl	string	Yes	The fileUrl returned from presign.
originalName	string	No	Original filename shown in the UI.
contentType	string	No	MIME type of the original file.

Code snippets

```
curl
curl -X POST http://localhost:4000/api/v1/images \
-H "Authorization: Bearer <token>" \
-H "Content-Type: application/json" \
-d '{"fileUrl":"/uploads/1714555200000-knee.png","originalName":"knee.jpg","contentType":"image/jpeg"}'
```

Response — 201 Created

```
json
{
  "id": "64f1a2b3c4d5e6f7a8b9c0d2",
  "fileUrl": "/uploads/1714555200000-knee.png",
  "originalName": "knee.jpg",
  "imageData": "data:image/png;base64,...",
  "uploadedBy": {
    "userId": "64f1a2b3c4d5e6f7a8b9c0d1",
    "email": "student@pace.edu"
  },
  "createdAt": "2026-05-01T10:05:00.000Z"
}
```

GET /api/v1/images

List all images uploaded by the current user. (Auth required)

Code snippets

```
curl
curl http://localhost:4000/api/v1/images \
-H "Authorization: Bearer <token>"
```

Response — 200 OK

```
json
[ { id, fileUrl, originalName, contentType, imageData, uploadedBy, createdAt }, ... ]
```

DELETE /api/v1/images/:imageId

Delete an image, its predictions, and its chat history. (Auth required)

Cascades automatically. Removing an image removes all predictions and chat histories linked to it.

Path parameter

Param	Type	Description
:imageId	string	MongoDB ObjectId of the image.

Code snippets

```
curl
curl -X DELETE http://localhost:4000/api/v1/images/64f1a2b3c4d5e6f7a8b9c0d2 \
-H "Authorization: Bearer <token>"
```

Response — 200 OK

```
json
{
  "ok": true,
  "deletedImageCount": 1,
  "deletedPredictionCount": 1,
  "deletedChatHistoryCount": 1
}
```

4. Predictions

Calls the ML service to classify the uploaded image and stores the result in MongoDB. If a prediction already exists for that image and user, the cached result is returned immediately — no second inference call is made.

POST /api/v1/predict

Run KL-grade inference on an uploaded image. (Auth required)

Request body

Field	Type	Required	Description
fileUrl	string	Yes	The /uploads/... path from image registration.
originalName	string	No	Original filename passed to the ML service.

Code snippets

```
curl
curl -X POST http://localhost:4000/api/v1/predict \
-H "Authorization: Bearer <token>" \
-H "Content-Type: application/json" \
-d '{"fileUrl":"/uploads/1714555200000-knee.png","originalName":"knee.jpg"}'
```

Response — 200 OK

```
json
{
  "id": "64f1a2b3c4d5e6f7a8b9c0d3",
  "imageId": "64f1a2b3c4d5e6f7a8b9c0d2",
  "grade": "Grade 2",
  "confidence": 87.43,
  "severityLabel": "Mild",
  "summary": "The model predicts Grade 2 with 87.43% confidence.",
  "probabilities": [
    {
      "label": "Grade 0",
      "value": 1.2
    },
    {
      "label": "Grade 1",
      "value": 3.5
    },
    {
      "label": "Grade 2",
      "value": 87.43
    },
    {
      "label": "Grade 3",
      "value": 6.1
    },
    {
      "label": "Grade 4",
      "value": 1.77
    }
  ],
  "heatmapUrl": "data:image/png;base64,...",
  "similarCases": "[ ... top 5 reference cases ... ]"
}
```

GET /api/v1/predictions

List all saved predictions for the current user. (Auth required)

Results are sorted newest first. Each item contains the full ML result nested under the result field.

Code snippets

```
curl
curl http://localhost:4000/api/v1/predictions \
-H "Authorization: Bearer <token>"
```

Response — 200 OK

```
json
[ { id, imageId, imageUrl, result: { grade, confidence, ... }, createdAt }, ... ]
```

GET /api/v1/predictions/export/csv

Download all predictions as a CSV file. (Auth required)

Returns a UTF-8 CSV with BOM for Excel compatibility. Columns: predictionId, imageId, imageUrl, grade, confidence, severityLabel, summary, createdAt.

Code snippets

```
curl
curl http://localhost:4000/api/v1/predictions/export/csv \
-H "Authorization: Bearer <token>" \
-o predictions.csv
```

5. Similar Cases

POST /api/v1/similar

Find visually similar reference cases. (Auth required)

Sends the image to the ML service, which extracts EfficientNet-B4 feature embeddings and runs cosine similarity against the SQLite reference database. Returns the top 5 matches with clinical metadata and base64 thumbnail. Optionally filters by KL grade.

Request body

Field	Type	Required	Description
fileUrl	string	Yes	The /uploads/... path of the image to compare.
grade	string	No	Filter by grade, e.g. "Grade 2". Omit to search all grades.

Code snippets

```
curl
curl -X POST http://localhost:4000/api/v1/similar \
-H "Authorization: Bearer <token>" \
-H "Content-Type: application/json" \
-d '{"fileUrl":"/uploads/1714555200000-knee.png","grade":"Grade 2"}'
```

Response — 200 OK

```
json
{
  "similarCases": [
    {
      "caseId": "OAI_001",
      "similarity": 0.9412,
      "klGrade": 2,
      "datasetSource": "OAI",
      "osteophyteSeverity": "Moderate",
      "jointSpaceNarrowing": "Mild",
      "subchondralSclerosis": "Present",
      "boneTexture": "Normal",
      "affectedCompartment": "Medial",
      "overallFindings": "Mild medial compartment OA with early osteophytes",
      "imageBase64": "data:image/png;base64,..."
    }
  ]
}
```

6. AI Chat

Conversational interface powered by Llama 4 Scout (17B) via Groq. The backend builds a system prompt from the prediction result and similar cases, then forwards the conversation to Groq. The X-ray image is attached to the first user message so the model can visually inspect it. Max tokens: 1024. Temperature: 0.4.

POST /api/v1/chat

Send a message to the radiology assistant. (Auth required)

Request body

Field	Type	Required	Description
imageBase64	string	Yes	Base64 data URI or /uploads/... path.
result	object	Yes	Full prediction object from /api/v1/predict.
similarCases	array	No	Similar cases from the prediction result.
messages	array	Yes	Full conversation history [{role, content}].

Code snippets

```
curl -X POST http://localhost:4000/api/v1/chat \
-H "Authorization: Bearer <token>" \
-H "Content-Type: application/json" \
-d '{
  "imageBase64": "/uploads/1714555200000-knee.png",
  "result": { "grade": "Grade 2", "confidence": 87.43, ... },
  "similarCases": [...],
  "messages": [{ "role": "user", "content": "What does Grade 2 mean?" }]
}'
```

Response — 200 OK

```
json
{
  "reply": "Grade 2 (Mild) osteoarthritis indicates definite osteophytes and possible joint space narrowing. Conservative management is typically recommended at this stage..."
}
```

POST /api/v1/chat-history

Save or update conversation history for a prediction. (Auth required)

Upserts the full message array. Call after each assistant reply to persist state.

Request body

Field	Type	Required	Description
predictionId	string	Yes	MongoDB ObjectId of the associated prediction.
messages	array	Yes	Full conversation array [{role, content}].

Code snippets

```
curl
curl -X POST http://localhost:4000/api/v1/chat-history \
-H "Authorization: Bearer <token>" \
-H "Content-Type: application/json" \
-d '{"predictionId":"64f...", "messages":[{"role":"user", "content":"..."}]}'
```

Response — 200 OK

```
json
{
  "ok": true
}
```

GET /api/v1/chat-history/:predictionId

Retrieve saved conversation history. (Auth required)

Path parameter

Param	Type	Description
:predictionId	string	MongoDB ObjectId of the prediction.

Code snippets

```
curl
curl http://localhost:4000/api/v1/chat-history/64f1a2b3c4d5e6f7a8b9c0d3 \
-H "Authorization: Bearer <token>"
```

Response — 200 OK (empty array if no history saved yet)

```
json
{
  "messages": [
    {
      "role": "user",
      "content": "What does Grade 2 mean?"
    },
    {
      "role": "assistant",
      "content": "Grade 2 indicates..."
    }
  ]
}
```

7. Quiz

The quiz uses a three-step flow. First fetch a random question — the KL grade is intentionally withheld. Show the user the X-ray image and let them select a grade. Then submit their answer to get the correct grade and a clinical explanation. Finally, save the final score to populate the leaderboard.

GET /api/v1/quiz/question

Get a random knee X-ray question. (Auth required)

The correct KL grade is not included in the response. It is only returned after the user submits via /quiz/submit.

Query parameter

Param	Type	Description
difficulty	string	easy (Grades 0 & 4), medium (all, default), hard (Grades 1–3).

Code snippets

```
curl
curl "http://localhost:4000/api/v1/quiz/question?difficulty=medium" \
-H "Authorization: Bearer <token>"
```

Response — 200 OK

```
json
{
  "caseId": "OAI_042",
  "imageBase64": "data:image/png;base64,...",
  "osteophyteSeverity": "Moderate",
  "jointSpaceNarrowing": "Moderate",
  "subchondralSclerosis": "Present",
  "boneTexture": "Coarsened",
  "affectedCompartment": "Medial",
  "overallFindings": "Moderate OA findings"
}
```

POST /api/v1/quiz/submit

Submit an answer and reveal the correct grade. (Auth required)

Request body

Field	Type	Required	Description
caseId	string	Yes	caseId from the question response.
userAnswer	integer	Yes	Selected KL grade: 0, 1, 2, 3, or 4.
difficulty	string	No	Difficulty level (stored for leaderboard context).
timeLeft	number	No	Seconds remaining when submitted.

Code snippets

```
curl
curl -X POST http://localhost:4000/api/v1/quiz/submit \
-H "Authorization: Bearer <token>" \
-H "Content-Type: application/json" \
-d '{"caseId":"OAI_042","userAnswer":3,"difficulty":"medium"}'
```

Response — 200 OK

```
json
{
  "correct": true,
  "correctGrade": 3,
  "osteophyteSeverity": "Moderate",
  "jointSpaceNarrowing": "Moderate",
  "subchondralSclerosis": "Present",
  "boneTexture": "Coarsened",
  "affectedCompartment": "Medial",
  "overallFindings": "Moderate OA with definite joint space narrowing"
}
```

POST /api/v1/quiz/score

Save the final quiz score. (Auth required)

Request body

Field	Type	Required	Description
score	number	Yes	Points scored.
total	number	Yes	Total possible points.
difficulty	string	Yes	Difficulty level played.
accuracy	number	Yes	Percentage correct (0–100).

Code snippets

```
curl
curl -X POST http://localhost:4000/api/v1/quiz/score \
-H "Authorization: Bearer <token>" \
-H "Content-Type: application/json" \
-d '{"score":850,"total":1000,"difficulty":"medium","accuracy":85.0}'
```

Response — 200 OK

```
json
{
  "ok": true
}
```

GET /api/v1/quiz/leaderboard

Get the top 10 scores for a difficulty. (Auth required)

Query parameter

Param	Type	Description
difficulty	string	easy medium hard. Defaults to medium.

Code snippets

```
curl
curl "http://localhost:4000/api/v1/quiz/leaderboard?difficulty=medium" \
-H "Authorization: Bearer <token>"
```

Response — 200 OK

```
json
[
  {
    "rank": 1,
    "email": "top@pace.edu",
    "score": 950,
    "total": 1000,
    "accuracy": 95,
    "difficulty": "medium",
    "createdAt": "2026-05-01T14:30:00.000Z"
  }
]
```

8. ML Service (Direct)

The ML service can also be called directly without going through the backend. This is useful during development or for integration testing. Note that when called from the backend, authentication is handled by the backend — the ML service itself has no auth layer.

GET /health

Health check. (No auth)

```
curl
curl http://localhost:8000/health
```

Response — 200 OK

```
json
{
  "status": "ok"
}
```

POST /predict

Run KL-grade classification and Grad-CAM generation. (No auth)

Accepts multipart/form-data. Only PNG and JPEG are accepted. Returns the full prediction result including similar cases from the reference database.

Form field

Field	Type	Required	Description
file	file	Yes	PNG or JPEG image.

Code snippets

```
curl
curl -X POST https://YOUR_HF_USERNAME-kneevision-ml.hf.space/predict \
-F "file=@knee_xray.jpg"
```

Response — 200 OK

```
json
{
  "grade": "Grade 2",
  "confidence": 87.43,
  "severityLabel": "Mild",
  "summary": "The model predicts Grade 2 with 87.43% confidence.",
  "probabilities": [
    {
      "label": "Grade 0",
      "value": 1.2
    },
    {
      "label": "Grade 2",
      "value": 87.43
    },
    {
      "label": "Grade 4",
      "value": 1.77
    }
  ],
  "heatmapUrl": "data:image/png;base64,...",
  "similarCases": "[ ... ]"
}
```

POST /similar

Find similar reference cases. (No auth)

Extracts EfficientNet-B4 embeddings from the uploaded image and runs cosine similarity against the reference database. Optionally filter by KL grade with the `kl_grade` query parameter.

Form field + optional query param

Field	Type	Required	Description
file	file	Yes	PNG or JPEG image.
kl_grade	integer	No	Filter to a specific KL grade (0–4). Query param, not form field.

Code snippets

```
curl
curl -X POST "https://YOUR_HF_USERNAME-kneevision-ml.hf.space/similar?kl_grade=2" \
-F "file=@knee_xray.jpg"
```

Response — 200 OK

```
json
{
  "similarCases": [
    {
      "caseId": "OAI_001",
      "similarity": 0.9412,
      "klGrade": 2,
      "overallFindings": "Mild medial compartment OA",
      "imageBase64": "data:image/png;base64,..."
    }
  ]
}
```

9. Quick Reference

Method	Endpoint	Auth	Description
POST	/api/v1/auth/register	No	Register new user
POST	/api/v1/auth/login	No	Login, receive JWT
POST	/api/v1/uploads/presign	No	Step 1 — get upload URL
PUT	/api/v1/uploads/:key	No	Step 2 — upload file bytes
POST	/api/v1/images	Yes	Step 3 — register image in DB
GET	/api/v1/images	Yes	List user images
DELETE	/api/v1/images/:imageId	Yes	Delete image + cascade
POST	/api/v1/predict	Yes	Run inference (cached)
GET	/api/v1/predictions	Yes	List saved predictions
GET	/api/v1/predictions/export/csv	Yes	Export predictions as CSV
POST	/api/v1/similar	Yes	Find similar reference cases
POST	/api/v1/chat	Yes	AI radiology chat
POST	/api/v1/chat-history	Yes	Save chat history
GET	/api/v1/chat-history/:predictionId	Yes	Get chat history
GET	/api/v1/quiz/question	Yes	Random quiz question
POST	/api/v1/quiz/submit	Yes	Submit quiz answer
POST	/api/v1/quiz/score	Yes	Save quiz score
GET	/api/v1/quiz/leaderboard	Yes	Top 10 leaderboard
GET	/health (ML)	No	ML service health
POST	/predict (ML)	No	Direct ML inference
POST	/similar (ML)	No	Direct similarity search